

1 Abstract

This report evaluates the efficacy of a data analysis workflow designed to quantify the baseline noise floor and derive the Limit of Detection (LOD) for UV chromatography signals. The workflow employed dynamic gradient stability thresholding to isolate baseline regions, followed by noise characterization and false positive rate (FPR) simulation. While the visual analysis of noise distributions suggests Gaussian behavior and successful baseline correction, a critical methodological failure was identified in the initial data sanitization phase. The dynamic thresholding algorithm failed to identify any stable regions across the analyzed runs, resulting in a zero-count for stable data points. Consequently, the subsequent LOD derivation and validation, which reported a 'Pass' status with an LOD of 1174.82 mAU, are based on invalid assumptions or default parameters rather than empirical baseline data. This report details the discrepancy between the visual noise characterization and the underlying data integrity failure, concluding that the current LOD values are unreliable for determining true detection limits.

2 Introduction

The primary objective of this data analysis was to programmatically establish the Limit of Detection (LOD) for UV signals (UV_1_280_mAU and UV_2_260_mAU) within a high-frequency chromatography dataset. The dataset presented significant challenges, including physically impossible negative signal values indicating baseline offsets, corrupted metadata columns, and a lack of explicit stage definitions for approximately 76% of the data rows. To address these issues, a three-step analytical plan was executed: (1) Data Sanitization and Dynamic Baseline Region Identification, (2) Noise Characterization and LOD Derivation, and (3) False Positive Rate Simulation. The methodology relied on inferring stable baseline regions by analyzing the derivative of the gradient concentration ('Conc_B_%') and filtering for null fraction IDs. The ultimate goal was to differentiate between random Gaussian noise and systematic drift, calculate a 3 LOD on baseline-corrected signals, and validate this threshold against a target false positive rate of less than 1% using synthetic peak injection.

3 Methodology

The analysis began with data sanitization, where columns ending in '_ml' (excluding 'volume_ml') were discarded, and rows with non-positive volume values were filtered to remove equilibration artifacts. Step 1 involved calculating the rolling derivative of 'Conc_B_%' to define a dynamic threshold (95th percentile of low-derivative values). A 'stable_region_mask' was generated where the absolute derivative fell below this threshold and 'Fraction_unique_ID' was null. Continuity checks were applied to ensure volume progression integrity. Step 2 utilized these stable regions to extract UV signals, apply median-based baseline correction, and calculate the raw standard deviation (.raw). Autocorrelation functions (ACF) were computed to detect drift, and the LOD was derived as 3.raw. Finally, Step 3 involved a simulation where synthetic Gaussian noise and peaks of inferred width were generated to test the LOD, adjusting the multiplier if the false positive rate exceeded 1%.

4 Results and Discussion

The visual analysis presented in Figure 1 (noise_analysis_all_runs.png) depicts the outcomes of the noise characterization phase. The figure displays a grid of subplots, each containing an Autocorrelation Function (ACF) plot and a histogram of the baseline-corrected noise distribution. Visually, the histograms appear centered near zero, suggesting that the median subtraction for baseline correction was effective in removing the raw negative offsets observed in the data. The symmetry of these distributions supports the assumption of normality required for the 3 LOD calculation. Furthermore, the ACF plots show correlations decaying within the 95% confidence intervals, which typically indicates the absence of significant systematic drift and the presence of random Gaussian noise. However, a critical contradiction arises when cross-referencing these visualizations with the markdown analysis reports from Step 1. The text-based results indicate that for

all analyzed runs, the ‘rows_in_stable_regions’ and ‘stable_region_mask_count’ were zero, and the ‘Continuity_Flag’ was False. This implies that the dynamic thresholding logic failed to isolate any valid baseline data points from the over one million rows in the dataset. Consequently, the noise distributions and LOD values visualized in Figure 1 cannot be derived from the intended stable regions. The reported LOD of 1174.82 mAU and the ‘Pass’ status in the FPR simulation (0.25% FPR) are therefore methodologically unsound, likely resulting from the algorithm defaulting to global statistics or invalid parameters due to the empty stable region mask. While Figure 1 suggests a robust noise floor, the underlying data extraction failure renders these visual results and the derived LOD unreliable for scientific interpretation.



Figure 1: Multi-panel visualization of Autocorrelation Functions (ACF) and baseline-corrected noise histograms with calculated LOD overlays for multiple chromatography runs.

5 Conclusion

In conclusion, while the visual outputs of the noise characterization phase (Figure 1) appear consistent with a Gaussian noise model and successful baseline correction, the foundational step of identifying stable baseline regions failed completely. The dynamic thresholding algorithm did not isolate any data points, leading to a situation where the calculated LOD and subsequent validation metrics are based on invalid inputs rather than empirical baseline data. The reported LOD of 1174.82 mAU is likely an artifact of this failure and does not represent the true detection limit of the system. Future iterations of this analysis must address the logic used for dynamic thresholding, potentially by adjusting the derivative calculation parameters or the criteria for ‘Fraction_unique.ID’ nullity, to ensure that stable regions are correctly identified before proceeding to noise quantification and LOD derivation.

A Appendix

A.1 A: Data Analysis Output Figures



Figure 2: noise_analysis_all_runs.png

A.2 B: Code Files

```
import pandas as pd
import numpy as np
import os

# Define file paths
input_file = '/inputs/chromatography_combined.csv'
output_report = '/workspace/analysis_report.md'

# Step 1: Load data
df = pd.read_csv(input_file)

# Explicitly cast 'Conc_B_%' and 'volume_ml' to float to handle mixed types
# Non-numeric values will be coerced to NaN to prevent runtime errors in math
# operations
df['Conc_B_%'] = pd.to_numeric(df['Conc_B_%'], errors='coerce')
df['volume_ml'] = pd.to_numeric(df['volume_ml'], errors='coerce')

# Step 2: Drop columns ending in '_ml' except 'volume_ml'
cols_to_drop = [col for col in df.columns if col.endswith('_ml') and col != 'volume_ml']
df = df.drop(columns=cols_to_drop)

# Step 3: Filter rows where 'volume_ml' <= 0
```

```

# Capture total rows per run BEFORE filtering to satisfy Plan requirement for '
    total_rows'
total_rows_per_run = df.groupby('run_no').size()
df = df[df['volume_ml'] > 0]

# Step 4: Calculate rolling derivative of 'Conc_B_%'
df['Conc_B_derivative'] = df['Conc_B_%'].diff()

# Step 5: Define dynamic threshold per run (95th percentile of low-derivative
    values within each run)
# Low-derivative values are those where absolute derivative is small (e.g., bottom
    50%)
def calculate_run_threshold(series):
    low_derivative_mask = series.abs() < series.abs().quantile(0.5)
    if low_derivative_mask.sum() == 0:
        return np.nan
    return series[low_derivative_mask].abs().quantile(0.95)

# Efficiently apply threshold calculation per run without redundant copies
# Explicitly select the column before transform to pass a Series to the function
df['run_dynamic_threshold'] = df.groupby('run_no')['Conc_B_derivative'].transform(
    calculate_run_threshold)

# Step 6: Define 'stable_region_mask'
# Treat 'Fraction_unique_ID' as null if it contains specific placeholder values
placeholder_values = ['N/A', 'NA', 'null', 'NULL', '']
df['Fraction_unique_ID'] = df['Fraction_unique_ID'].replace(placeholder_values, np
    .nan)

stable_region_mask = (
    (df['Conc_B_derivative'].abs() < df['run_dynamic_threshold']) &
    (df['Fraction_unique_ID'].isna())
)
df['stable_region_mask'] = stable_region_mask

# Step 7: Verify 'volume_ml' continuity within each run
df['volume_diff'] = df['volume_ml'].diff()
# Calculate median difference within each 'run_no' group
df['run_median_diff'] = df.groupby('run_no')['volume_diff'].transform('median')
continuity_flag = (df['volume_diff'].abs() <= 2 * df['run_median_diff'])

# Apply continuity check only within stable regions
df['Continuity_Flag'] = df['stable_region_mask'] & continuity_flag

# Step 8: Group stable regions by 'run_no' and generate summary
# 'total_rows' is the pre-filter count (captured before Step 3)
# 'rows_after_cleaning' is the post-filter count
# 'rows_in_stable_regions' is the count of rows meeting the stable mask criteria
# 'stable_region_mask_count' is explicitly added as a separate column per Plan
    requirements
summary_df = df.groupby('run_no').agg(
    rows_after_cleaning=('run_no', 'size'),
    rows_in_stable_regions=('stable_region_mask', 'sum'),
    stable_region_mask_count=('stable_region_mask', 'sum'),
    continuity_flag_count=('Continuity_Flag', 'sum'),
    Conc_B_derivative_threshold=('run_dynamic_threshold', 'first')
).reset_index()

# Merge the pre-filter total rows count

```

```

summary_df = summary_df.merge(
    total_rows_per_run.reset_index(name='total_rows'),
    on='run_no',
    how='left'
)

# Ensure Continuity_Flag is boolean
summary_df['Continuity_Flag'] = summary_df['continuity_flag_count'] > 0
summary_df = summary_df.drop(columns=['continuity_flag_count'])

# Step 9: Append summary to 'analysis_report.md' (limit to 10 lines)
with open(output_report, 'a') as f:
    f.write("### Step 1: Data Sanitization and Dynamic Baseline Region Identification\n")
    f.write("Summary Table:\n")
    # Efficient string truncation for report
    summary_str = summary_df.to_string(index=False)
    summary_lines = summary_str.split('\n')
    if len(summary_lines) > 10:
        f.write('\n'.join(summary_lines[:10]))
        f.write("\n... (truncated)")
    else:
        f.write(summary_str)
    f.write("\n")

print("Analysis complete. Report updated.")

```

Listing 1: Code_1

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
from scipy.signal import correlate
import os
import logging

# Configure logging for validation warnings
logging.basicConfig(level=logging.WARNING, format='%(levelname)s: %(message)s')

# Ensure output directory exists
os.makedirs('/workspace', exist_ok=True)

# Load data
file_path = '/inputs/chromatography_combined.csv'
df = pd.read_csv(file_path)

# Debug: Inspect data before failing
if 'chromatography_stage' in df.columns:
    print('Unique chromatography_stage values:', df['chromatography_stage'].unique())
else:
    print('Column chromatography_stage not found. Available columns:', df.columns.tolist())

# Select required variables
cols = ['run_no', 'UV_1_280_mAU', 'UV_2_260_mAU', 'stable_region_mask', 'volume_ml', 'System_flow_ml_min']

```

```

# Check if stable_region_mask exists, if not, derive it or raise error
if 'stable_region_mask' not in df.columns:
    # Attempt to derive from chromatography_stage if available
    if 'chromatography_stage' in df.columns:
        target_stage = None
        if 'FT' in df['chromatography_stage'].values:
            target_stage = 'FT'
        elif 'W1' in df['chromatography_stage'].values:
            target_stage = 'W1'

        if target_stage:
            df['stable_region_mask'] = (df['chromatography_stage'] == target_stage)
            .astype(int)
        else:
            raise ValueError(f"Column 'stable_region_mask' is missing and no suitable stable stage ('FT' or 'W1') found in 'chromatography_stage'. Available values: {df['chromatography_stage'].unique()}")
    else:
        raise ValueError("Column 'stable_region_mask' is missing and no chromatography_stage column found to derive it.")

df = df[cols].copy()

# Filter for stable regions with validation
mask_col = df['stable_region_mask']
if mask_col.isna().any():
    logging.warning("NaN values found in 'stable_region_mask'. Excluding these rows.")
    df = df.dropna(subset=['stable_region_mask'])
    mask_col = df['stable_region_mask']

valid_mask_values = mask_col.isin([0, 1])
if not valid_mask_values.all():
    invalid_count = (~valid_mask_values).sum()
    logging.warning(f"{invalid_count} non-binary integer values found in 'stable_region_mask'. Excluding these rows.")
    df = df[valid_mask_values]

df_stable = df[df['stable_region_mask'] == 1].copy()

# Group by run_no
runs = df_stable['run_no'].unique()

# Dynamic grid calculation based on number of runs
n_runs = len(runs)
n_cols = 4
n_rows = (n_runs + n_cols - 1) // n_cols

# Constants
TARGET_WIDTH_MINUTES = 1.0

# Prepare figure with shared axes as per feedback
fig, axes = plt.subplots(n_rows, n_cols, figsize=(20, 15), sharex=True, sharey=True)

# Handle case where there is only 1 row or 1 column (axes is not 2D)
if n_rows == 1 and n_cols == 1:
    axes = np.array([[axes]])
elif n_rows == 1:
    axes = axes.reshape(1, -1)

```

```

elif n_cols == 1:
    axes = axes.reshape(-1, 1)

axes = axes.flatten()

# Pre-filter data to avoid inefficient iteration inside the loop
run_data_dict = {run_id: df_stable.loc[df_stable['run_no'] == run_id] for run_id
    in runs}

for i, run_id in enumerate(runs):
    if i >= len(axes):
        break

    run_data = run_data_dict[run_id]

    if run_data.empty:
        continue

    # 1. Extract signals
    sig_280 = run_data['UV_1_280_mAU'].values
    sig_260 = run_data['UV_2_260_mAU'].values
    vol = run_data['volume_ml'].values
    flow = run_data['System_flow_ml_min'].values

    # 2. Baseline Correction
    median_280 = np.median(sig_280)
    median_260 = np.median(sig_260)

    noise_280 = sig_280 - median_280
    noise_260 = sig_260 - median_260

    # 3. Calculate sigma_raw
    sigma_280_raw = np.std(noise_280, ddof=1)
    sigma_260_raw = np.std(noise_260, ddof=1)

    # 4. Check skewness/kurtosis and Box-Cox
    skew = stats.skew(noise_280)
    kurt = stats.kurtosis(noise_280)

    transformed_noise = noise_280.copy()
    sigma_final = sigma_280_raw
    box_cox_applied = False

    if skew > 1 or kurt > 3:
        min_val = np.min(noise_280)
        shift = abs(min_val) + 1e-6
        shifted_noise = noise_280 + shift

        try:
            transformed_shifted, lam = stats.boxcox(shifted_noise)
            transformed_noise = transformed_shifted - shift
            sigma_final = np.std(transformed_noise, ddof=1)
            box_cox_applied = True
            # Feedback: Add logging when box_cox_applied is True
            logging.info(f"Run_{run_id}: Box-Cox transformation applied. Sigma
                recalculated.")
        except ValueError:
            pass

```

```

# 5. Compute ACF
n = len(transformed_noise)
acf = correlate(transformed_noise, transformed_noise, mode='full')
acf = acf[n-1:]
acf = acf / acf[0]
ci = 1.96 / np.sqrt(n)

# 6. LOD
lod = 3 * sigma_final

# 7. Rows per ml and Volume Progression Error
diff_vol = np.diff(vol)
positive_diffs = diff_vol[diff_vol > 0]

volume_progression_error = False
rows_per_ml = 0

if len(positive_diffs) > 0:
    ml_per_row = np.median(positive_diffs)
    if ml_per_row > 0:
        rows_per_ml = 1.0 / ml_per_row
    else:
        # Feedback: Add logging when rows_per_ml is 0 due to lack of volume
        # progression
        logging.warning(f"Run_{run_id}: Median volume difference is 0. Setting
            rows_per_ml to 0. Inferred peak width will be 0.")
        rows_per_ml = 0

    if len(positive_diffs) < 0.10 * len(vol):
        volume_progression_error = True
else:
    # Feedback: Add logging when rows_per_ml is 0 due to lack of volume
    # progression
    logging.warning(f"Run_{run_id}: No positive volume differences found.
        Setting rows_per_ml to 0. Inferred peak width will be 0.")
    rows_per_ml = 0
    volume_progression_error = True

# 8. Inferred peak width and Flow Variability
median_flow = np.median(flow)
flow_std = np.std(flow, ddof=1)
flow_variability = (flow_std / median_flow) if median_flow > 0 else 0
flag_flow_var = flow_variability > 0.10

inferred_width_rows = rows_per_ml * TARGET_WIDTH_MINUTES * median_flow

# Plotting
ax = axes[i]

# Feedback: Remove twinx() and plot ACF and Histogram on the same axes with
# shared Y-axes
# We will plot ACF (line) and Histogram (bars) on the same axis.
# To distinguish, we use different colors and line styles.
# Note: ACF Y is correlation (-1 to 1), Histogram Y is count.
# Since we cannot share Y-axes meaningfully if scales differ wildly, we
# normalize histogram or accept visual overlap.
# However, the feedback explicitly says "shared Y-axes". We will plot them on
# the same axis.
# To make it readable, we might need to scale the histogram or accept that

```



```

    they share the scale.
# Given the feedback "shared Y-axes", we plot both on #ax#.

# ACF Plot
lags = np.arange(len(acf))
ax.plot(lags, acf, label='ACF', color='blue', alpha=0.7, linewidth=1.5)
ax.axhline(y=ci, color='gray', linestyle='--', label='95% CI')
ax.axhline(y=-ci, color='gray', linestyle='--')
ax.set_ylabel('Correlation/Count', color='black') # Shared Y label
ax.grid(True, alpha=0.3)

# Histogram Plot
# We plot the histogram on the same axis.
# To make it visible alongside ACF (which is -1 to 1), we might need to
    normalize the histogram or just plot it.
# The feedback says "shared Y-axes", so we do not use twinx.
# We use density=False (default) for count, but if counts are huge, it will
    dwarf ACF.
# Let's assume the user wants to see the shape. We will plot it as is.
# To distinguish, we use a different color and alpha.
ax.hist(transformed_noise, bins=50, alpha=0.6, color='orange', label='Noise_
    Dist', density=False)
ax.axvline(x=lod, color='red', linestyle='-', linewidth=2, label=f'LOD({lod
    :.4f})')
ax.axvline(x=-lod, color='red', linestyle='-', linewidth=2)

# Title
ax.set_title(f"Run_{run_id}\nLOD: {lod:.4f}\nInferred_Width: {
    inferred_width_rows:.2f}_rows", fontsize=10)

# Legend with flags and inferred_peak_width
from matplotlib.lines import Line2D
# Feedback: Modify legend to include 'inferred_peak_width' value
flag_handles = [
    Line2D([0], [0], color='black', marker='o', linestyle='None', label=f'
        Flow_Variability_Flag: {flag_flow_var}'),
    Line2D([0], [0], color='black', marker='s', linestyle='None', label=f'
        Volume_Progression_Error: {volume_progression_error}'),
    Line2D([0], [0], color='black', marker='^', linestyle='None', label=f'
        Inferred_Peak_Width: {inferred_width_rows:.2f}_rows')
]

lines, labels = ax.get_legend_handles_labels()
all_lines = lines + flag_handles
all_labels = labels + [h.get_label() for h in flag_handles]

ax.legend(all_lines, all_labels, loc='upper_right', fontsize=8)

# Adjust layout
plt.tight_layout()

# Save figure
plt.savefig('/workspace/noise_analysis_all_runs.png', dpi=150, bbox_inches='tight'
)
plt.close()

print("Analysis complete. Saved to /workspace/noise_analysis_all_runs.png")

```

Listing 2: Code_2

```

import pandas as pd
import numpy as np
from scipy.signal import find_peaks
import os
import logging

# Configure logging
logging.basicConfig(level=logging.INFO, format='%(levelname)s: %(message)s')

# Configuration
INPUT_FILE = "/inputs/chromatography_combined.csv"
OUTPUT_FILE = "/workspace/analysis_report.md"
TARGET_FPR = 1.0 # 1% (Target FPR as percentage)
MAX_ITERATIONS = 20

# 1. Load Data
cols_to_load = ['run_no', 'UV_1_280_mAU', 'UV_2_260_mAU']
df_headers = pd.read_csv(INPUT_FILE, nrows=0)
available_cols = df_headers.columns.tolist()

cols_needed = ['run_no', 'UV_1_280_mAU', 'UV_2_260_mAU']
derived_cols = ['inferred_peak_width', 'LOD', '_raw']

for col in derived_cols:
    if col in available_cols:
        cols_needed.append(col)

df = pd.read_csv(INPUT_FILE, usecols=cols_needed)

# Check for 'chromatography_stage' and 'Stable' stage
if 'chromatography_stage' in df.columns:
    stable_mask = df['chromatography_stage'] == 'Stable'
    if not stable_mask.any():
        raise ValueError("No suitable 'Stable' stage found in chromatography_stage; derivation failed.")
    df = df[stable_mask].reset_index(drop=True)
else:
    logging.warning("Column 'chromatography_stage' not found. Proceeding without stage filtering.")

# Filter out rows with missing critical data
df = df.dropna(subset=['UV_1_280_mAU'])

if df.empty:
    raise ValueError("No valid data found after dropping NaNs.")

# Extract statistics
baseline_mean = df['UV_1_280_mAU'].mean()

# Handle _raw
box_cox_applied = False
if '_raw' in df.columns:
    sigma_raw = df['_raw'].mean()
else:
    logging.info("_raw column missing. Calculating raw standard deviation from UV_1_280_mAU (no Box-Cox transformation applied).")
    sigma_raw = df['UV_1_280_mAU'].std()
    if sigma_raw <= 0:

```

```

        sigma_raw = 0.01

if sigma_raw <= 0:
    raise ValueError("_raw _must_be_positive.")

# Handle inferred_peak_width
if 'inferred_peak_width' in df.columns:
    peak_width_fwhm = df['inferred_peak_width'].mean()
    logging.info(f"Using_inferred_peak_width_from_data:{peak_width_fwhm}")
else:
    logging.warning("Column 'inferred_peak_width' missing. Using default peak_
        width_of_10.0_rows.")
    peak_width_fwhm = 10.0

if peak_width_fwhm <= 0:
    raise ValueError("inferred_peak_width_must_be_positive.")

# Feedback: Use inferred_peak_width directly as peak_sigma (no division by 2.355)
peak_sigma = peak_width_fwhm
logging.info(f"Derived_peak_sigma:{peak_sigma}(directly_from_inferred_peak_width
)")

# Feedback: Raise error if peak_sigma is invalid, do not set to 0.0
if peak_sigma <= 0:
    raise ValueError(f"Calculated_peak_sigma_is_{peak_sigma}.Invalid_simulation_
        parameter.")

# Handle LOD
if 'LOD' in df.columns:
    lod_base = df['LOD'].mean()
else:
    lod_base = 3.0 * sigma_raw

# 2. Simulation Loop
current_multiplier = lod_base / sigma_raw if sigma_raw > 0 else 3.0
validated_multiplier = current_multiplier
final_fpr = 100.0
validation_status = "Fail"

# Simulation parameters
noise_length = 10000
num_synthetic_peaks = 50
num_simulations = 100

for iteration in range(MAX_ITERATIONS):
    current_lod_threshold = validated_multiplier * sigma_raw

    total_false_positives = 0
    total_true_positives = 0

    for _ in range(num_simulations):
        # Generate synthetic Gaussian noise (loc=0)
        noise = np.random.normal(loc=0, scale=sigma_raw, size=noise_length)
        signal = baseline_mean + noise

        # Random positions for peaks
        min_idx = int(peak_sigma * 3)
        max_idx = noise_length - int(peak_sigma * 3)
        if max_idx <= min_idx:

```

```

        continue

valid_positions = np.arange(min_idx, max_idx)
if len(valid_positions) < num_synthetic_peaks:
    num_synthetic_peaks = len(valid_positions)

peak_indices = np.random.choice(valid_positions, size=num_synthetic_peaks,
                                replace=False)

# Feedback: Vectorized Peak Injection using broadcasting
# Pre-allocate array for peak contributions
peak_contributions = np.zeros(noise_length)
amplitudes = np.random.uniform(0.5, 2.0, size=num_synthetic_peaks) *
    current_lod_threshold

# Pre-calculate Gaussian kernel
window_size = int(3 * peak_sigma) + 1
x_local = np.arange(-window_size, window_size + 1)
gaussian_values = np.exp(-0.5 * (x_local / peak_sigma) ** 2)

# Efficiently inject peaks using vectorized bounds handling
# Create a mask for valid indices within bounds for each peak
# We iterate over peaks but use vectorized math for the slice to avoid
    inner loops over x

# Calculate start and end indices for all peaks
starts = np.maximum(0, peak_indices - window_size)
ends = np.minimum(noise_length, peak_indices + window_size + 1)

# Calculate local start and end indices relative to the kernel
# local_start = start - (idx - window_size)
local_starts = starts - (peak_indices - window_size)
# local_end = end - (idx - window_size)
local_ends = ends - (peak_indices - window_size)

# Apply peaks
for i, idx in enumerate(peak_indices):
    start = starts[i]
    end = ends[i]
    local_start = local_starts[i]
    local_end = local_ends[i]

    # Vectorized slice addition
    peak_contributions[start:end] += amplitudes[i] * gaussian_values[
        local_start:local_end]

signal += peak_contributions

# Apply peak detection
detected_peaks, _ = find_peaks(signal, height=baseline_mean +
    current_lod_threshold)

# Vectorized Matching
if len(detected_peaks) == 0:
    continue

dist_matrix = np.abs(detected_peaks[:, None] - peak_indices[None, :])
threshold_dist = 1.5 * peak_sigma

```

```

        is_matched = (dist_matrix < threshold_dist).any(axis=1)

        matched_count = np.sum(is_matched)
        false_positives_count = len(detected_peaks) - matched_count

        total_true_positives += matched_count
        total_false_positives += false_positives_count

    # Feedback: Re-evaluate FPR calculation.
    # FPR = (total_false_positives / (num_simulations * noise_length)) * 100
    current_fpr = (total_false_positives / (num_simulations * noise_length)) * 100

    # Feedback: Log if box_cox_applied is True
    if box_cox_applied:
        logging.info("Box-Cox transformation was applied. Sigma was recalculated accordingly.")

    if current_fpr < TARGET_FPR:
        final_fpr = current_fpr
        validation_status = "Pass"
        break
    else:
        validated_multiplier += 0.1

# Final Calculations
final_lod_mau = validated_multiplier * sigma_raw

# 3. Output Generation
output_text = f"""
### Step 3: False Positive Rate Simulation and Validation
- **Final LOD Multiplier**: {validated_multiplier:.2f}
- **Calculated LOD (MAU)**: {final_lod_mau:.4f}
- **Simulated False Positive Rate (%)**: {final_fpr:.6f}
- **Synthetic Peak Width (rows)**: {peak_sigma:.2f}
- **Validation Status**: {validation_status}
"""

os.makedirs(os.path.dirname(OUTPUT_FILE), exist_ok=True)
with open(OUTPUT_FILE, 'a') as f:
    f.write(output_text)

print("Step 3 completed. Results appended to analysis_report.md")

```

Listing 3: Code_3